The Business User Is the Developer Now

Most enterprise software is bad, and we have made our peace with the wrong explanation for why. We blame the vendor, the budget, the ageing stack. The real reason is more uncomfortable: the software is bad because it was built for everyone, which is another way of saying it was built for no one in particular.

Think about how an internal tool actually gets made. Forty people need something. They need forty slightly different things. So the requirements get gathered, averaged, negotiated, and trimmed until what survives is a grey compromise that offends nobody and delights nobody. The person who actually does the work — who knows exactly which three fields matter and which report they open every Monday morning — has almost no say in the result. Their need was real and specific. It arrived at the backlog, got generalised into a "use case," and waited. The backlog is where individual need goes to be averaged into mush.

We accepted this because building software was expensive. When every feature costs weeks of engineering time, you cannot build a version for each person, so you build one version for the mean and call it efficiency. Consensus was never really a value. It was a budget constraint wearing the costume of a value. And that constraint is dissolving.

The inversion

For thirty years we have organised software work around a simple split: developers write, operations run, and the whole discipline of DevOps was about shortening the distance between the two. I think the next version flips the roles entirely. The developer is no longer the person who writes the code. It is the person who has the need.

Here is the move. A business user — someone in finance, in claims, in logistics, who has never written a line of anything — describes the application they want in plain language. Not a ticket. A conversation. An AI assistant sits with them and does what a good business analyst does: asks the awkward questions, surfaces the edge cases they had not considered, separates what they said from what they meant, and turns a vague want into an actual specification. The user is the developer. IT, as we will see, becomes operations. And nobody has to win an argument with thirty-nine other people before anything gets built.

How it actually works

If this sounds like science fiction, it should not, because the hard half already ships. In modern software teams the loop runs today: someone files a request in near-plain language, hands it to an agent, and the agent plans the work, makes the changes, runs the tests, and returns a finished draft for a human to review and approve. The better versions of this even bake in a quiet, important rule — the person who asked for the work cannot be the person who signs it off. The machine does the labour; a second human owns the judgement.

What changes in the model I am describing is who gets to start that loop. Not just the engineers. Anyone.

So follow a request through. The need becomes a spec. The spec becomes a blueprint — and this is the part that matters — and the blueprint is not just the app. It carries everything the user neither sees nor should have to think about: the security tests, the data-access rules, the compliance checks, the audit trail. These are not a gate the request slams into at the end. They are compiled in from the start, the way a good builder pours the footings before raising the walls. If the system can detect a problem, it can also propose the fix, inside the pipeline, before anything ships. The business user never learns what a SQL injection is, and never should. That knowledge lives in the platform, not in their head.

Then the blueprint goes to operations. And here is the second inversion: operations is not a room full of people reading your request. It is a platform, and its rules are written as code. A request that stays inside the guardrails — standard data, known patterns, small blast radius — is simply accepted, automatically. The ones that do not — the request that wants customer financial data, or reaches into a regulated system — are routed to a human, because those are the only ones a person's attention is actually worth spending

on. The reply you get is not a ticket number that vanishes into a queue. It is a sentence: we are building this for you, it will be ready Thursday.

You do not switch this on at full power. You climb to it. First the system only reads — it answers questions and pulls data, proving it can be trusted before it is allowed to touch anything. Then it assists — it proposes, a human confirms. Only then does it automate — the things it gets right every single time stop interrupting a human at all, and the exceptions become the only thing people look at. Autonomy is earned one rung at a time, and earned fastest where the cost of being wrong is lowest.

Why this is not just letting everyone loose

Now the objection, because it is a good one. Is this not simply handing everybody a toy that lets them build their own little app, and will that not end in chaos?

It will — if you do the lazy version. Code produced quickly by people who cannot read it has a measurable habit of opening more security holes, leaking more secrets, and getting waved through review precisely because the green checkmark is reassuring and everyone is in a hurry. You can improvise your way to a working app in an afternoon. Can you improvise your way through debugging it at two in the morning when it is mishandling someone's payroll? Ten thousand bespoke apps with no shared floor beneath them is not liberation. It is a maintenance swamp and a breach waiting to be named in a headline — a pile of local wins that never add up to anything the company can bank.

The escape is the unglamorous part nobody wants to fund. The platform owns the things that must be true for every app — identity, data contracts, policy, logging — so that no individual app gets to reinvent them or quietly skip them. The security checks are a hard gate, not an honour system. The business owns the outcome; the platform owns the floor that outcome stands on. And — this is the one most organisations get wrong — the team racing to ship must never be the same team that owns the guardrails, or the guardrails always lose. The reassuring truth underneath all of this is that most of the work is not exotic. It is ordinary, disciplined systems engineering pointed at a new kind of input. The genuinely new problems — that the system is non-deterministic, that it can be confidently wrong, that it has to be evaluated continuously rather than tested once — are real, but they are a small and knowable slice, not a reason to keep everyone waiting in line.

What IT becomes

Which tells you what the IT department actually becomes. It is not the office that says no, and it is not the backlog that swallows ambition and hands it back as a roadmap. It becomes a platform team whose product is other people's ability to build safely. That is the whole job. You stop measuring it by tickets closed and start measuring it by how quickly a safe application reaches the person who needed it — and by how rarely that person had to ask permission for the obvious.

The last era of corporate technology was about taking the things people already did and putting them on a screen. The next one is about letting the people closest to the work build the tools that fit that work, and trusting the platform — not the committee — to keep it safe. "Developer" was never meant to be a job title. It was meant to be whoever had the problem.